

Scone Tutorial

A Step-by-Step Introduction to the
Scone Knowledge-Base System

A companion to the *Scone User's Guide*

April 22, 2026

This tutorial is a companion to the *Scone User's Guide* and walks a new user through building a small knowledge base from an empty Scone image. Scone was developed by Scott E. Fahlman and his group at Carnegie Mellon University's Language Technologies Institute. The examples in this tutorial are designed to run against the open-source Scone release maintained at <https://github.com/blakemcbride/scone>.

The engine and core knowledge bases are copyright © 2003–2014 Carnegie Mellon University and are distributed under the Apache License, Version 2.0.

Contents

1	About This Tutorial	5
2	Starting Scone	7
3	Elements and Namespaces	9
4	The Is-A Hierarchy	11
5	Listing and Showing	13
6	Roles	15
7	Statements	17
8	Splits and Disjointness	19
9	Cancellation	21
10	Contexts	23
11	Saving Your Work	25
12	Where to Go Next	27

1 About This Tutorial

This is a hands-on introduction to Scone. It assumes you can read and type simple Common Lisp expressions at a REPL, but it does not assume any prior experience with knowledge-base systems or with marker-passing inference. Each chapter introduces one idea, walks through a short example at the REPL, and ends with a small exercise that extends the example. Later chapters build on the knowledge base constructed in earlier ones, so the best way to use this tutorial is linearly, typing along as you go.

What you will need.

- A Common Lisp implementation — the examples are tested with Steel Bank Common Lisp (SBCL), but any of CMUCL, CCL, LispWorks, or Allegro should work.
- A checkout of the Scone source tree, with **scone-loader.lisp** at the top level.
- A terminal window running your Lisp, from which you can read and type. That is all.

A note on conventions. Lines the reader should type at the REPL are shown with a leading asterisk, like this:

```
* (new-type {animal} {thing})
```

Lines without an asterisk are either output produced by Scone or Lisp text that appears inside a multi-line expression. Internal element names are written in the Scone **{curly-brace}** notation throughout.

What this tutorial is not. It is not an encyclopedia of the Scone API. When you need the signature of a function or a list of all available predicates, turn to the *Scone User's Guide*. Similarly, it is not a textbook on knowledge representation; for the deeper theory behind Scone's design, see Scott E. Fahlman, *NETL: A System for Representing and Using Real-World Knowledge*, MIT Press, 1979.

2 Starting Scone

Open a terminal in the top-level directory of your Scone checkout and start your Lisp. If you are using SBCL, the interaction will look like this:

```
$ sbcl
This is SBCL 2.x.x, an implementation of ANSI Common Lisp.
...
* (load "scone-loader.lisp")

;;; Call (scone) to start Scone.
T
```

The **scone-loader.lisp** file is a small convenience script that locates **engine.lisp** and the **kb/** directory relative to itself. Calling **(scone)** loads the engine and then loads the bootstrap knowledge base, which installs the handful of elements that the engine itself refers to:

```
* (scone)
Loading KB "bootstrap".
Assigning 0 as context marker, 1 as activation marker.
31 deferred connections fixed, 0 remain.
Load of "bootstrap" completed. 68 elements total.
```

At this point Scone is running, but it only knows about the 68 primitive elements installed by **bootstrap.lisp**. To get common-sense knowledge about animals, people, places, and so on, load the **core** knowledge base on top:

```
* (load-kb "core")
Loading KB "core".
Loading KB "core-components/upper".
...
Load of "core" completed. 3613 elements total.
```

The core KB is a bundle of eleven files covering upper ontology, units, space, time, simple physics, biology, information, social structure, the person model, episodic memory, and geopolitics. Loading it populates Scone with about 3,600 elements — the raw material every subsequent chapter builds on.

A first sanity check. With **core** loaded, ask Scone whether a dog is an animal:

```
* (is-x-a-y? {dog} {animal})
:YES
```

The answer **:YES** confirms that the is-a hierarchy from the core KB is in place.

Stopping. Scone has no explicit shutdown — just exit your Lisp (**(sb-ext:exit)** in SBCL, or **(quit)** in most other implementations). The in-memory KB is discarded. To preserve work done in a session, use **checkpoint-kb**, covered in Chapter 11.

Exercise. Confirm that a **{cat}** is an **{animal}**, and that a **{rock}** is *not* an **{animal}**. What does **is-x-a-y?** return in the second case?

3 Elements and Namespaces

Everything in a Scone KB is an *element*. The top of the type hierarchy is the built-in element **{thing}**: every other element is, directly or indirectly, a kind of **{thing}**.

Elements come in two main flavors. A *type* represents a category — **{dog}**, **{animal}**, **{furniture}**. An *individual* represents a specific thing — the particular dog named Rover, the Empire State Building, yesterday. In code you create them with **new-type** and **new-indv** respectively.

Your first element. Let's build a small zoo. First, make a new namespace so that our tutorial elements don't collide with the core KB's vocabulary:

```
* (in-namespace "zoo")
#<namespace "zoo">
```

A *namespace* is just a named dictionary that keeps internal names distinct. All subsequent **new-type** and **new-indv** calls will register their elements in the **"zoo"** namespace. Now create a type and two individuals:

```
* (new-type {dog} {animal})
{zoo:dog}
* (new-indv {Rover} {dog})
{zoo:Rover}
* (new-indv {Whiskers} {cat})
{zoo:Whiskers}
```

Each call returns the new element, printed in curly-brace notation. Because ***print-namespace-in-elements*** defaults to **:maybe**, elements from a namespace other than the current one (for example **{animal}**, which lives in **common**) are printed with their namespace prefix the first time they appear.

Looking up elements. Given a name, **lookup-element** returns the element it refers to, or **nil** if none exists:

```
* (lookup-element {Rover})
{zoo:Rover}
* (lookup-element {Fluffy})
NIL
```

Most Scone functions take elements *or* element-inames (the **{name}** literal syntax is itself an iname that gets resolved on demand), so you rarely need to call **lookup-element** yourself. It is useful when you have a bare string and want to convert it to an element.

Exercise. Create another individual **{Fido}** as a **{dog}**. Then, without looking, guess what **(is-x-a-y? {Fido} {animal})** will return — and verify at the REPL.

4 The Is-A Hierarchy

The backbone of a Scone KB is its *is-a hierarchy*: a web of parent-of and instance-of relationships that says how every element relates to the types above it. **{dog}** is a kind of **{mammal}**; **{mammal}** is a kind of **{animal}**; Rover *is* a **{dog}**. That chain of statements is what lets Scone conclude that Rover is an animal without anyone having said so directly.

Declaring relationships. When you create a new type or individual, the **parent** argument automatically puts it into the is-a hierarchy. If you need to add another parent later, use **new-is-a**:

```
* (new-is-a {Rover} {pet})
{zoo:is-a-12345}
```

Rover is now simultaneously a **{dog}** and a **{pet}**. Scone returns the newly-created IS-A link element; the internal name is auto-generated and rarely interesting.

Querying the hierarchy. The predicate **is-x-a-y?** returns one of three values:

- **:YES** — it is definitely the case that **x** is a **y** in the current context.
- **:NO** — it is definitely *not* the case that **x** is a **y**, and asserting otherwise would cause a contradiction.
- **:MAYBE** — the KB has no definite knowledge either way.

```
* (is-x-a-y? {Rover} {mammal})
:YES
* (is-x-a-y? {Rover} {furniture})
:MAYBE
```

The second result illustrates Scone’s open-world assumption: unless something in the KB rules out Rover being furniture, the answer is **:MAYBE**, not **:NO**. We’ll see how to make exclusions explicit in Chapter 8.

Siblings: EQ and NOT-EQ. Alongside **new-is-a**, Scone supports **new-eq** (identifying two elements as the same thing) and **new-not-eq** (asserting they are distinct). Use **new-eq** when you discover that two independently-created elements refer to the same entity — for example, “the mayor” turns out to be the same person as “Jane Smith.”

Exercise. Create a type **{working-dog}** as a subtype of **{dog}**, then create **{Lassie}** as a **{working-dog}**. Confirm that **(is-x-a-y? {Lassie} {mammal})** returns **:YES**.

5 Listing and Showing

When a knowledge base grows beyond a handful of elements, looking at it in bulk becomes important. Scone provides three families of functions for this:

- **list-** functions return a Lisp list of matching elements. Good for programmatic use.
- **show-** functions print a human-readable report to the ***show-stream*** (by default standard output). Good for browsing at the REPL.
- **mark-** functions propagate markers over the KB. These are the underlying primitives; most users rarely call them directly.

What kinds of dogs are there?

```
* (list-inferiors {dog})  
({zoo:Lassie} {zoo:Fido} {zoo:Rover} ...)
```

list-inferiors returns every element reachable downward from its argument — both subtypes and instances. There’s also a **show-inferiors** that prints the result in a readable form:

```
* (show-inferiors {dog})
```

If you want only one or the other, use **list-subtypes** / **show-subtypes** (type nodes only) or **list-instances** / **show-instances** (individuals only).

Surveying the ontology. **show-ontology** prints the full is-a tree below a given element:

```
* (show-ontology :under {mammal} :n 20)
```

The **:under** keyword restricts the report to the subtree rooted at **{mammal}**. The **:n** keyword caps the output at the given number of elements, which is handy when you aren’t sure how big the subtree is.

English names. Scone maintains a two-way dictionary between elements and English words. **show-synonyms** takes a string and prints every element currently known by that name:

```
* (show-synonyms "dog")
```

You may see more than one element if the core KB has entries for, say, the verb “to dog” alongside the noun.

Exercise. Print the ontology under **{bird}** with **:n 15**, and try to pick out the difference between a “type” line and an “indv” line in the output.

6 Roles

So far the only fact we can state about an element is its type. But most useful knowledge is relational: *Rover's owner is Alice*, *Rover's color is brown*. Scone represents relationships like these with *roles*.

A role is a named slot attached to a type. When you say “every dog has an owner,” what you are really doing is giving the type **{dog}** an **owner** role whose filler must be a **{person}**. Every instance of **{dog}** automatically inherits the role; filling it gives a specific fact about a specific dog.

Two kinds of role. **new-indv-role** creates a role that takes a *single* filler — “a dog has (one) owner.” **new-type-role** creates a role that takes *many* fillers — “a dog has (any number of) siblings.”

```
* (new-indv-role {owner} {dog} {person})
{zoo:owner}
* (new-type-role {sibling} {dog} {dog})
{zoo:sibling}
```

Both calls take three arguments: the role’s name, the *owner* type (the kind of thing that *has* the role), and the *filler* type (the kind of thing that the role’s value must be).

Filling a role. To say who a specific dog’s owner is, use **x-is-the-y-of-z**:

```
* (new-indv {Alice} {person})
{zoo:Alice}
* (x-is-the-y-of-z {Alice} {owner} {Rover})
{zoo:eq-54321}
```

Read that as “Alice is the owner of Rover.” Scone returns the underlying EQ-link (again, the name is auto-generated).

Now you can ask questions:

```
* (the-x-of-y {owner} {Rover})
{zoo:Alice}
```

Roles are first-class elements. The role **{owner}** is itself a Scone element. That means you can subclass it, cancel it, or put meta-information on it using the same tools you’d use for any other concept — there is no separate “role language.”

Exercise. Give Whiskers the cat an owner **{Bob}**. Then verify that **(the-x-of-y {owner} {Whiskers})** returns **{Bob}** and that **(the-x-of-y {owner} {Rover})** still returns **{Alice}**.

7 Statements

A *statement* is a stand-alone assertion that two elements are related in a particular way. Where a role attaches a relation to a type (every dog has an owner), a statement attaches a relation to specific elements (Rover chased Whiskers yesterday).

Creating a relation. Before you can state that Rover chased Whiskers, you need a relation object for “chased.” Relations are created with **new-relation**:

```
* (new-relation {chases} :a-inst-of {animal}
                  :b-inst-of {animal})
{zoo:chases}
```

The **:a-inst-of** and **:b-inst-of** keywords constrain the allowed arguments — both the chaser and the chased must be animals. Trying to state that a rock chased something would be flagged as incompatible.

Making a statement. Once the relation exists, **new-statement** asserts it of specific elements:

```
* (new-statement {Rover} {chases} {Whiskers})
{zoo:Rover-chases-Whiskers}
```

Asking about statements. The predicate **statement-true?** answers yes/no questions, and **list-rel** / **show-rel** enumerate the things an element is related to:

```
* (statement-true? {Rover} {chases} {Whiskers})
T
* (list-rel {chases} {Rover})
({zoo:Whiskers})
```

list-rel returns the *b* side of every **chases** statement whose *a* side is Rover. Its mirror **list-rel-inverse** runs the query in the other direction: “everything that chases Whiskers.”

Exercise. Define a relation **{likes}** between two **{animal}**s and state that Whiskers likes Alice (hint: you will first need to make sure **{Alice}** is, or can be, a kind of **{animal}** — or define **{likes}** more broadly). Query the statement with **statement-true?** to confirm.

8 Splits and Disjointness

Earlier we saw that `is-x-a-y?` returned `:MAYBE` when asked whether Rover is a piece of furniture. That was because nothing in the KB had said “nothing can be both a dog and a piece of furniture.” *Splits* are how you say that.

A split declares that a set of types are pairwise disjoint: no element can be two of them at once. A *complete* split adds the further claim that *everything* under some common parent *must* be one of them. For example, every person is either male or female (a complete split under `{person}`), but an animal might be a bird, mammal, reptile, fish, or something else — that’s a (plain) split, because not every split-member is enumerated.

Creating a split.

```
* (new-split '({mammal} {bird} {reptile} {fish}))
{zoo:Split-87654}
```

Now Scone knows these four types are incompatible with each other. Ask the same furniture question as before:

```
* (is-x-a-y? {Rover} {furniture})
:MAYBE
```

Still `:MAYBE`, because `{furniture}` is not in any split with the types Rover belongs to. But now try:

```
* (is-x-a-y? {Rover} {bird})
:NO
```

Scone can now *rule out* Rover being a bird, because Rover is known to be a mammal and the split says no element can be both.

Pre-fighting a new link. Before actually creating a link that might cause a contradiction, you can ask `incompatible?`:

```
* (incompatible? {Rover} {bird})
{mammal}
```

The non-nil return identifies the ancestor that would clash (Rover is a mammal, mammals and birds are split). If the proposed link would be fine, `incompatible?` returns `nil`.

Exercise. Define a complete split under `{pet}` between `{dog}` and `{cat}`, then confirm that Scone reports `:NO` when you ask whether Rover is a cat.

9 Cancellation

Splits say “these categories never overlap.” Cancellation is the reverse: it lets one specific element *opt out* of a fact that it would otherwise inherit. Classic example: birds fly, but penguins don’t.

Setting up the default.

```
* (new-statement {bird} {can-do} {flying})
{common:bird-can-do-flying}
```

Every bird now inherits this fact. Confirm with a subtype:

```
* (new-type {robin} {bird})
* (statement-true? {robin} {can-do} {flying})
T
```

Adding an exception. To say penguins are the exception, cancel the inherited statement for **{penguin}**:

```
* (new-type {penguin} {bird})
* (new-cancel {penguin} <the statement above>)
```

In practice you identify the inherited statement by the element it was bound to, then pass that element to **new-cancel** as the second argument. The function signature is **(new-cancel a b &key :context ...)**: everything below **a** has link **b** cancelled.

After the cancel-link is in place:

```
* (statement-true? {penguin} {can-do} {flying})
NIL
* (statement-true? {robin} {can-do} {flying})
T
```

The inherited default still holds for other birds; it is only cancelled for penguins.

Cancel or context? Cancellation is local and permanent. If you need an exception that is only true in one situation — say, “in this fictional story, penguins *can* fly” — you want a *context* instead. Contexts are the subject of the next chapter.

Exercise. Cancel the **{owner}** role on a type **{stray-dog}** (which should be a subtype of **{dog}**). Then confirm that **(the-x-of-y {owner} {some-stray})** returns no owner.

10 Contexts

Up to now every fact you’ve added has been true in “reality.” But knowledge bases often need to represent beliefs, stories, hypotheses, or past and future states — situations in which some facts differ from the baseline. Scone handles this with *contexts*.

A context is a node that groups a set of facts. Every link and every statement in Scone is implicitly tied to a context via its context-wire. When you ask a query, Scone only considers facts that are active in the current context (or in contexts inherited by it).

The default context. When you start Scone, the current context is **{general}** — a descendant of the universal context **{universal}**. All the zoo facts you’ve added so far live in **{general}**.

Creating a branching context. Suppose you want to reason about a children’s-book world in which penguins *do* fly. Make a new context whose parent is the general context, so it inherits everything, then switch to it:

```
* (new-context {story-world} {general})
{zoo:story-world}
* (in-context {story-world})
{zoo:story-world}
```

Now add the fact that is true only in the story:

```
* (new-statement {penguin} {can-do} {flying})
{zoo:penguin-can-do-flying}
```

This statement is attached to **{story-world}**, not to **{general}**. In the story context, the earlier cancellation is overridden:

```
* (statement-true? {penguin} {can-do} {flying})
T
```

Switch back to reality and the cancellation takes effect again:

```
* (in-context {general})
{general}
* (statement-true? {penguin} {can-do} {flying})
NIL
```

Contexts form a hierarchy. Contexts are is-a’d to their parent contexts. A query in **{story-world}** inherits everything true in **{general}**, then layers the story’s own facts on top. This makes contexts a natural tool for episodic memory, belief modeling, simulation, and alternate-history reasoning.

Exercise. Create a context **{stone-age}** whose parent is **{general}**, switch to it, and assert that **{electricity}** does not exist (hint: use **new-is-not-a** or a cancel-link against a relevant parent). Query the same fact in **{general}** and in **{stone-age}** and observe the difference.

11 Saving Your Work

A Scone session is a live Lisp image. When you exit Lisp, the KB is gone. To persist the knowledge you've built, *checkpoint* it to a file.

Checkpointing everything.

```
* (checkpoint-kb "zoo-snapshot")
Checkpointed KB to #P".../kb/zoo-snapshot.lisp".
```

checkpoint-kb writes every non-bootstrap element currently in memory to a plain Lisp source file. If you don't specify a path, the file goes under ***default-kb-pathname*** (set by **scone-loader.lisp** to your repo's **kb/** directory).

Reloading the snapshot. In a fresh Scone session, load the snapshot on top of **bootstrap** and **core**:

```
* (load "scone-loader.lisp")
* (scone)
* (load-kb "core")
* (load-kb "zoo-snapshot")
Loading KB "zoo-snapshot".
Load of "zoo-snapshot" completed.  N elements total.
```

Your zoo is back. Because the snapshot is plain text, you can also version-control it, hand-edit it, or share it.

Checkpointing just the new stuff. If your session built on top of **core**, and you only want to save the work you did yourself, use **checkpoint-new** instead:

```
* (checkpoint-new "zoo-additions")
```

This writes out only the elements created since the last **load-kb** completed — usually exactly the ones you typed. The resulting file is smaller and loads faster.

Exercise. Checkpoint the KB, then exit SBCL, start a fresh Scone, reload your snapshot, and confirm that **(is-x-a-y? {Rover} {mammal})** still returns **:YES**.

12 Where to Go Next

You now know enough Scone to build a small KB, query it, and persist it. The next steps depend on what you want to do with the system.

For depth on the API. The *Scone User's Guide* (in the same **manual/** directory as this tutorial) is the reference work. It documents every function and variable in the engine, organized by subsystem. In particular, look at:

- *Scone Elements* and *Adding Elements to the Scone KB* for the full set of element-creation functions (**new-type**, **new-indv**, **new-map**, **new-relation**, and their many keyword arguments).
- *Markers* and *Marker Scans* for the low-level primitives underneath **list-inferiors**, **is-x-a-y?**, and the rest. If you ever need to write a custom query that isn't already provided, these are the tools you'll reach for.
- *Consistency Checking* for the **find-split-violations** / **incompatible?** / **find-path** family — useful for auditing a large KB.

For the theory. Scone descends from Scott E. Fahlman's earlier NETL system. The book-length treatment is his 1979 MIT Press volume *NETL: A System for Representing and Using Real-World Knowledge*. The ideas behind the marker-passing, parallel-inheritance style of knowledge representation — the core of why Scone is shaped the way it is — are laid out there. Scott Fahlman's "Knowledge Nuggets" blog at <http://www.cs.cmu.edu/~nuggets/> contains more recent essay-length commentary on the Scone design and its applications.

For broader lexical coverage. The **kb/lexical-components/** directory contains WordNet-derived knowledge bases that can be layered on top of **core**. These files are large and slow to load, but if you want Scone to understand a wide vocabulary of English words, (**load-kb "lexical-components/wordnet-concepts"**) is the place to start.

For your own application. The intended model is that you write your own KB file (just a Lisp source file using **new-type**, **new-indv**, etc.) and load it on top of **core**. Once your KB is in place, Scone is just another library your application calls.

That's the full tour. Happy reasoning.